```
#include <windows.h>
#include <d3d11.h>
#include <d3dcompiler.h>
#include <cmath> // ИСПРАВЛЕНИЕ: Подключили математические функции sqrt и pow для CPU
#include <commctrl.h>                    // Системная библиотека элементов управления Windows
#pragma comment(lib, "comctl32.lib")     // Автоматическая линковка библиотеки
#pragma comment(lib, "d3d11.lib")
#pragma comment(lib, "d3dcompiler.lib")

// === ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ ДЛЯ НЕЗАВИСИМОГО ДИАГНОСТИЧЕСКОГО ОКНА ===
HWND                g_hDlgWnd = NULL;          // Дескриптор диагностического окна Windows
HWND                g_hDlgStatusWnd = NULL;    // Строка статуса диагностического окна панели
IDXGISwapChain* g_pDlgSwapChain = NULL;    // Цепочка буферов для второго экрана
ID3D11RenderTargetView* g_pDlgRenderTargetView = NULL; // Цель отрисовки диагностического окна
void CreateDiagnosticWindow(HINSTANCE hInstance, HWND hParentWnd);
int                 g_diagMode = 1;            // Текущий режим панели: 1, 2, 3 или 4
ID3D11Texture2D* g_pMacroStaging = NULL;           // Промежуточная текстура для передачи данных из GPU в CPU
extern ID3D11Texture2D* g_pStagingCompressed;          // Промежуточная текстура для оригинала кадра

UINT* pPixelsBuffer = NULL;                    // Наш глобальный исходный массив кадра
UINT* pRawPixels = NULL;                       // Наш глобальный массив кадра для CPU и отладки!
UINT* pCompressedBuffer4x = NULL;                  // Буфер для хранения уменьшенной копии кадра на CPU
UINT* pStretchedBuffer = NULL;                 // Буфер для хранения кадра, растянутого обратно после сжатия

// === ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ ДЛЯ ПОЛОС ПРОКРУТКИ ===
int                 g_dlgScrollX = 0; // Сдвиг по горизонтали для окна диагностики
int                 g_dlgScrollY = 0; // Сдвиг по вертикали для окна диагностики
int                 g_dlgScrollX_4x4 = 0; // Сдвиг по горизонтали для окна диагностики
int                 g_dlgScrollY_4x4 = 0; // Сдвиг по вертикали для окна диагностики
```

```
extern UINT          g_currentImgWidth;  // Храним размеры текущего файла
extern UINT          g_currentImgHeight;

// ========================================================================
// НОВАЯ ФУНКЦИЯ ДИНАМИЧЕСКОЙ НАСТРОЙКИ ПОЛЗУНКОВ ПОД ТЕКУЩИЙ РЕЖИМ
// ========================================================================
void UpdateDlgScrollbars(HWND hWnd)
{
   if (g_currentImgWidth == 0 || g_currentImgHeight == 0) return;

   SCROLLINFO si = {};
   si.cbSize = sizeof(SCROLLINFO);
   si.fMask = SIF_RANGE | SIF_PAGE | SIF_POS;
   si.nMin = 0;

   // Получаем реальные размеры рабочей области отладочного окна прямо сейчас
   RECT rcClient;
   GetClientRect(hWnd, &rcClient);
   int winW = rcClient.right - rcClient.left;
   int winH = rcClient.bottom - rcClient.top; // Вычитаем 40 пикселей под меню "Режимы"

   if (winW <= 0) winW = 400;
   if (winH <= 0) winH = 320;

   si.nPage = winW;

   if (g_diagMode == 2) // Если выбран Режим 3: Сжатый в 8 раз
   {
      UINT compressedW = g_currentImgWidth / 8;
      UINT compressedH = g_currentImgHeight / 8;

      si.nPage = winW;
```

```
        si.nMax = compressedW;
        if (g_dlgScrollX_4x4 > (si.nMax - (int)si.nPage)) g_dlgScrollX_4x4 = (si.nMax > (int)si.nPage) ? (si.nMax - (int)si.nPage) : 0;
        si.nPos = g_dlgScrollX_4x4;
        SetScrollInfo(hWnd, SB_HORZ, &si, TRUE);

        si.nPage = winH; // Размер вертикальной страницы равен физической высоте окна winH
        si.nMax = compressedH;
        if (g_dlgScrollY_4x4 > (si.nMax - (int)si.nPage)) g_dlgScrollY_4x4 = (si.nMax > (int)si.nPage) ? (si.nMax - (int)si.nPage) : 0;
        si.nPos = g_dlgScrollY_4x4;
        SetScrollInfo(hWnd, SB_VERT, &si, TRUE);
    }
    else // Для Режимов 1 и 2 (Исходный и Монохромный)
    {
        // Исходные кадры отображаются с укрупнением в 2 раза
        si.nMax = g_currentImgWidth;
        if (g_dlgScrollX > (si.nMax - (int)si.nPage)) g_dlgScrollX = (si.nMax > (int)si.nPage) ? (si.nMax - (int)si.nPage) : 0;
        si.nPos = g_dlgScrollX;
        SetScrollInfo(hWnd, SB_HORZ, &si, TRUE);

        si.nPage = winH;
        si.nMax = g_currentImgHeight;
        if (g_dlgScrollY > (si.nMax - (int)si.nPage)) g_dlgScrollY = (si.nMax > (int)si.nPage) ? (si.nMax - (int)si.nPage) : 0;
        si.nPos = g_dlgScrollY;
        SetScrollInfo(hWnd, SB_VERT, &si, TRUE);
    }
}

void UpdateDiagnosticStatusText(HWND hWnd)
{
    if (!g_hDlgStatusWnd) return;
```

```
    wchar_t statusBuf[256];
    if (g_diagMode == 2) // Режим 3: Сжатый в 8 раз
    {
        wsprintf(statusBuf, L"Сжатый режим 8x8 | Позиция X: %d, Y: %d",
            g_dlgScrollX_4x4, g_dlgScrollY_4x4);
    }
    else // Режимы 1 и 2: Исходный и Монохромный
    {
        wsprintf(statusBuf, L"Режим 1:1 | Позиция X: %d, Y: %d",
            g_dlgScrollX, g_dlgScrollY);
    }

    // Отправляем текст в первую секцию строки статуса диагностического окна
    SendMessage(g_hDlgStatusWnd, SB_SETTEXT, 0, (LPARAM)statusBuf);
}



// === 1. ОКОННАЯ ПРОЦЕДУРА ДЛЯ ДИАГНОСТИЧЕСКОЙ ПАНЕЛИ ===
LRESULT CALLBACK DiagnosticWndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM
lParam)
{
    switch (message)
    {
    case WM_HSCROLL:
    {
        SCROLLINFO si = {};
        si.cbSize = sizeof(SCROLLINFO);
        si.fMask = SIF_ALL; // Запрашиваем абсолютно все параметры, включая nTrackPos
        GetScrollInfo(hWnd, SB_HORZ, &si);

        int oldPos = si.nPos;
```

```
// В Win32 при отпускании мыши после перетаскивания приходит код SB_THUMBPOSITION,
// и реальная позиция лежит строго в si.nTrackPos. Извлекаем её:
if (LOWORD(wParam) == SB_THUMBPOSITION || LOWORD(wParam) == SB_THUMBTRACK)
{
   si.nPos = si.nTrackPos;
}
else
{
   // Обработка кликов по стрелочкам и пустому полю ползунка
   switch (LOWORD(wParam))
   {
   case SB_LINELEFT:  si.nPos -= 10; break; // Удобный шаг сдвига на 10 пикселей
   case SB_LINERIGHT: si.nPos += 10; break;
   case SB_PAGELEFT:  si.nPos -= si.nPage; break;
   case SB_PAGERIGHT: si.nPos += si.nPage; break;
   }
}

// Жестко удерживаем бегунок в рамках доступного размера кадра
if (si.nPos < si.nMin) si.nPos = si.nMin;
if (si.nPos > (si.nMax - (int)si.nPage)) si.nPos = si.nMax - (int)si.nPage;

if (si.nPos != oldPos)
{
   // ИСПРАВЛЕНИЕ: строго записываем чистую позицию без каких-либо искажений масштаба
   if (g_diagMode == 2) {
      g_dlgScrollX_4x4 = si.nPos;
   }
   else {
      g_dlgScrollX = si.nPos; // Чистый сдвиг 1:1 для Режимов 1 и 2
   }
```

```
        si.fMask = SIF_POS;
        SetScrollInfo(hWnd, SB_HORZ, &si, TRUE); // Обновляем позицию бегунка на рамке
        // НОВЫЙ КОД: Обновляем координаты в строке статуса
        UpdateDiagnosticStatusText(hWnd);
        InvalidateRect(hWnd, NULL, FALSE);      // Вызываем принудительный WM_PAINT кадра
    }
    return 0;
}

// Обработка вертикальной прокрутки
case WM_VSCROLL:
{
    SCROLLINFO si = {};
    si.cbSize = sizeof(SCROLLINFO);
    si.fMask = SIF_ALL;
    GetScrollInfo(hWnd, SB_VERT, &si);

    int oldPos = si.nPos;

    if (LOWORD(wParam) == SB_THUMBPOSITION || LOWORD(wParam) == SB_THUMBTRACK)
    {
        si.nPos = si.nTrackPos;
    }
    else
    {
        // Обработка кликов по стрелочкам и пустому полю ползунка
        switch (LOWORD(wParam))
        {
        case SB_LINEUP:  si.nPos -= 10; break; // Удобный шаг сдвига на 10 пикселей
        case SB_LINEDOWN: si.nPos += 10; break;
        case SB_PAGEUP:  si.nPos -= si.nPage; break;
        case SB_PAGEDOWN: si.nPos += si.nPage; break;
```

```
        }
    }

    // Проверяем рамки диапазона
    if (si.nPos < si.nMin) si.nPos = si.nMin;
    if (si.nPos > (si.nMax - (int)si.nPage)) si.nPos = (si.nMax - (int)si.nPage);

    if (si.nPos != oldPos)
    {
        if (g_diagMode == 2) {
            g_dlgScrollY_4x4 = si.nPos; // Строго для Режима 3 (case 2) пишем в сжатую координату
        }
        else {
            g_dlgScrollY = si.nPos;     // Для Режимов 1 и 2 (case 0 и case 1) пишем в обычную координату
        }
        si.fMask = SIF_POS;
        SetScrollInfo(hWnd, SB_VERT, &si, TRUE); // Перерисовываем сам бегунок
        // НОВЫЙ КОД: Обновляем координаты в строке статуса
        UpdateDiagnosticStatusText(hWnd);
        InvalidateRect(hWnd, NULL, FALSE); // Заставляем окно перерисовать картинку
    }
    return 0;
  }

  case WM_SIZE:
    return 0; // Блокируем промежуточную перерисовку во время растягивания

  case WM_EXITSIZEMOVE:
    // Сначала принудительно обновляем размеры страниц ползунков под новую растянутую рамку окна!
    UpdateDlgScrollbars(hWnd);
```

```
        // Только после этого перерисовываем кадр в масштабе 1:1
        InvalidateRect(hWnd, NULL, FALSE);
        return 0;
    case WM_COMMAND:
    {
        int wmId = LOWORD(wParam);
        // Диапазон пунктов от 2000 (Режим 1) до 2007 (Режим 8)
        if (wmId >= 2000 && wmId <= 2007)
        {
            g_diagMode = wmId - 2000;
            // Получаем доступ к меню панели диагностики
            HMENU hMenuBar = GetMenu(hWnd);
            if (hMenuBar) {
                HMENU hPopupMenu = GetSubMenu(hMenuBar, 0); // Самое первое выпадающее подменю "Режимы"
                if (hPopupMenu) {
                    // Снимаем галочки со всех восьми режимов панели
                    for (int id = 2000; id <= 2007; id++) {
                        CheckMenuItem(hPopupMenu, id, MF_BYCOMMAND | MF_UNCHECKED);
                    }
                    // Ставим галочку на выбранный режим
                    CheckMenuItem(hPopupMenu, wmId, MF_BYCOMMAND | MF_CHECKED);
                }
            }
            wchar_t titleBuf[128];
            // Выводим физические размеры сжатых буферов прямо в заголовок для диагностики
            wsprintf(titleBuf, L"Режим %d | Буфер 4x4: %dx%d",
                g_diagMode + 1,
                g_currentImgWidth / 4,
                g_currentImgHeight / 4);

            SetWindowText(hWnd, titleBuf);
```

```
        UpdateDlgScrollbars(hWnd); // Перестраиваем диапазоны ползунков под выбранный режим кадра

        // УДАЛИЛИ ПРИНУДИТЕЛЬНЫЙ ТЯЖЕЛЫЙ ФЛАГ GPU, ЧТОБЫ УБРАТЬ 10-СЕКУНДНОЕ ЗАВИСАНИЕ ОКНА!

        InvalidateRect(hWnd, NULL, FALSE);
        return 0;
    }
  }
  break;

  case WM_PAINT:
  {
    PAINTSTRUCT ps;
    HDC hdc = BeginPaint(hWnd, &ps);

    // Берем только ту область, которую Windows официально просит обновить (ps.rcPaint)
    // Это предотвратит повторные циклы рисования поверх уже готового кадра
    HBRUSH hBrush = NULL;

    switch (g_diagMode)
    {
    case 0: // Режим 1: Исходный
    {
      // Очищаем фон ОДИН раз
      HBRUSH hBg = CreateSolidBrush(RGB(0, 0, 0));
      FillRect(hdc, &ps.rcPaint, hBg); // Очищаем строго область запроса, а не весь 400x400
      DeleteObject(hBg);

      if (pPixelsBuffer != NULL && g_currentImgWidth > 0 && g_currentImgHeight > 0)
      {
        // Прямо запрашиваем у Windows текущие размеры видимой страницы ползунков
```

```
SCROLLINFO siH = { sizeof(SCROLLINFO), SIF_POS | SIF_PAGE };
GetScrollInfo(hWnd, SB_HORZ, &siH);
SCROLLINFO siV = { sizeof(SCROLLINFO), SIF_POS | SIF_PAGE };
GetScrollInfo(hWnd, SB_VERT, &siV);

// В режиме 1:1 индексы считывания равны чистой позиции бегунка
UINT startX = siH.nPos;
UINT startY = siV.nPos;

// Граница перебора равна позиции старта плюс физическая ширина видимой страницы окна
UINT endX = startX + siH.nPage;
UINT endY = startY + siV.nPage;

if (endX > g_currentImgWidth) endX = g_currentImgWidth;
if (endY > g_currentImgHeight) endY = g_currentImgHeight;

for (UINT y = startY; y < endY; y++)
{
   UINT rowOffset = y * g_currentImgWidth;
   for (UINT x = startX; x < endX; x++)
   {
      DWORD color = pRawPixels[rowOffset + x];
      BYTE b = (BYTE)(color & 0xFF);
      BYTE g = (BYTE)((color >> 8) & 0xFF);
      BYTE r = (BYTE)((color >> 16) & 0xFF);

      HBRUSH hPxlBrush = CreateSolidBrush(RGB(r, g, b));
      RECT pxlRect = {
         (int)(x - startX),
         (int)(y - startY),
         (int)(x - startX + 1),
         (int)(y - startY + 1)
```

```
                };
                FillRect(hdc, &pxlRect, hPxlBrush);
                DeleteObject(hPxlBrush);
            }
        }
    }
}
break;
case 1: // Режим 2: Монохромный
{
    HBRUSH hBg = CreateSolidBrush(RGB(0, 0, 0));
    FillRect(hdc, &ps.rcPaint, hBg);
    DeleteObject(hBg);

    if (pPixelsBuffer != NULL && g_currentImgWidth > 0 && g_currentImgHeight > 0)
    {
        // Прямо запрашиваем у Windows текущие размеры видимой страницы ползунков
        SCROLLINFO siH = { sizeof(SCROLLINFO), SIF_POS | SIF_PAGE };
        GetScrollInfo(hWnd, SB_HORZ, &siH);
        SCROLLINFO siV = { sizeof(SCROLLINFO), SIF_POS | SIF_PAGE };
        GetScrollInfo(hWnd, SB_VERT, &siV);

        // В режиме 1:1 индексы считывания равны чистой позиции бегунка
        UINT startX = siH.nPos;
        UINT startY = siV.nPos;

        // Граница перебора равна позиции старта плюс физическая ширина видимой страницы
окна
        UINT endX = startX + siH.nPage;
        UINT endY = startY + siV.nPage;

        if (endX > g_currentImgWidth) endX = g_currentImgWidth;
        if (endY > g_currentImgHeight) endY = g_currentImgHeight;
```

```
        for (UINT y = startY; y < endY; y++)
        {
            UINT rowOffset = y * g_currentImgWidth;
            for (UINT x = startX; x < endX; x++)
            {
                DWORD color = pRawPixels[rowOffset + x];
                BYTE b = (BYTE)(color & 0xFF);
                BYTE g = (BYTE)((color >> 8) & 0xFF);
                BYTE r = (BYTE)((color >> 16) & 0xFF);

                float grayVal = (float)r * 0.299f + (float)g * 0.587f + (float)b * 0.114f;
                BYTE gray = (BYTE)grayVal;

                HBRUSH hPxlBrush = CreateSolidBrush(RGB(gray, gray, gray));
                RECT pxlRect = {
                    (int)(x - startX),
                    (int)(y - startY),
                    (int)(x - startX + 1),
                    (int)(y - startY + 1)
                };
                FillRect(hdc, &pxlRect, hPxlBrush);
                DeleteObject(hPxlBrush);
            }
        }
    }
}
break;
case 2: // Режим 3: Сжатый в 8 раз (ВЫВОД ЧЕРЕЗ БЫСТРЫЙ BITBLT)
{
    if (pCompressedBuffer4x != NULL && g_currentImgWidth > 0 && g_currentImgHeight > 0)
    {
```

```
UINT compressedW = g_currentImgWidth / 8;
UINT compressedH = g_currentImgHeight / 8;
int startX = g_dlgScrollX_4x4;
int startY = g_dlgScrollY_4x4;

RECT rcClient;
GetClientRect(hWnd, &rcClient);
int winW = rcClient.right - rcClient.left;
int winH = rcClient.bottom - rcClient.top;
if (winW <= 0 || winH <= 0) break;

// 1. Создаем временный контекст в памяти, совместимый с экраном панели
HDC hMemDC = CreateCompatibleDC(hdc);

// 2. Описываем структуру пикселей для создания совместимого холста (DIB)
BITMAPINFO bmi = {};
bmi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
bmi.bmiHeader.biWidth = compressedW;
bmi.bmiHeader.biHeight = -(int)compressedH; // Читаем массив сверху вниз
bmi.bmiHeader.biPlanes = 1;
bmi.bmiHeader.biBitCount = 32;
bmi.bmiHeader.biCompression = BI_RGB;

// 3. Создаем физический холст Windows HBITMAP на основе нашего массива пикселей
void* pBits = nullptr;
HBITMAP hBmp = CreateDIBSection(hdc, &bmi, DIB_RGB_COLORS, &pBits, NULL, 0);
if (hBmp && pBits)
{
   // Копируем данные из нашего буфера ОЗУ прямо в память холста Windows
   memcpy(pBits, pCompressedBuffer4x, compressedW * compressedH * sizeof(UINT));

   // 4. Выбираем холст в созданный контекст памяти
```

```
            HBITMAP hOldBmp = (HBITMAP)SelectObject(hMemDC, hBmp);

            // Очищаем фон окна под обрезку кадра
            HBRUSH hBg = CreateSolidBrush(RGB(0, 0, 0));
            FillRect(hdc, &rcClient, hBg);
            DeleteObject(hBg);

            // 5. МГНОВЕННОЕ И БЫСТРОЕ КОПИРОВАНИЕ БИТ В БИТ
            // Вычисляем, сколько пикселей физически помещается в окно
            int viewW = ((startX + winW) > (int)compressedW) ? (int)(compressedW - startX) : winW;
            int viewH = ((startY + winH) > (int)compressedH) ? (int)(compressedH - startY) : winH;

            BitBlt(
               hdc,     // Куда рисуем (экран панели отладки)
               0, 0,    // Левый верхний угол приемника
               viewW, viewH, // Размеры копируемого блока пикселей
               hMemDC,  // Откуда берем (наш контекст с макроструктурой)
               startX, startY, // Координаты смещения ползунков в источнике
               SRCCOPY  // Флаг прямого попиксельного переноса памяти
            );

            // 6. Наводим порядок в системе: возвращаем старый объект и удаляем холст
            SelectObject(hMemDC, hOldBmp);
            DeleteObject(hBmp);
         }

         // Удаляем временный контекст памяти
         DeleteDC(hMemDC);
      }
   }
   break;
```

```
    case 3: // Режим 4: Растянутый после сжатия (ВЫВОД ИЗ ЧЕСТНОГО ПОЛНОРАЗМЕРНОГО МАССИВА)
    {
        if (pStretchedBuffer != NULL && g_currentImgWidth > 0 && g_currentImgHeight > 0)
        {
            // Берем координаты из обычных ползунков 1:1, а не из сжатых 8x8!
            int startX = g_dlgScrollX;
            int startY = g_dlgScrollY;

            RECT rcClient;
            GetClientRect(hWnd, &rcClient);
            int winW = rcClient.right - rcClient.left;
            int winH = rcClient.bottom - rcClient.top;
            if (winW <= 0 || winH <= 0) break;

            HDC hMemDC = CreateCompatibleDC(hdc);

            BITMAPINFO bmi = {};
            bmi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
            // === ИСПРАВЛЕНИЕ: ПРИНУДИТЕЛЬНО ЗАДАЕМ ПОЛНЫЙ ИСХОДНЫЙ РАЗМЕР КАДРА ===
            bmi.bmiHeader.biWidth = g_currentImgWidth;   // Строго полный размер (4128)
            bmi.bmiHeader.biHeight = -(int)g_currentImgHeight; // Строго полный размер (3096) с минусом
            bmi.bmiHeader.biPlanes = 1;
            bmi.bmiHeader.biBitCount = 32;
            bmi.bmiHeader.biCompression = BI_RGB;

            void* pBits = nullptr;
            HBITMAP hBmp = CreateDIBSection(hdc, &bmi, DIB_RGB_COLORS, &pBits, NULL, 0);
            if (hBmp && pBits)
            {
                // Скоростное копирование памяти полноразмерного слепка
                memcpy(pBits, pStretchedBuffer, g_currentImgWidth * g_currentImgHeight * sizeof(UINT));
```

```
            HBITMAP hOldBmp = (HBITMAP)SelectObject(hMemDC, hBmp);

            HBRUSH hBg = CreateSolidBrush(RGB(0, 0, 0));
            FillRect(hdc, &rcClient, hBg);
            DeleteObject(hBg);

            int viewW = ((startX + winW) > (int)g_currentImgWidth) ? (int)(g_currentImgWidth - startX) :
winW;
            int viewH = ((startY + winH) > (int)g_currentImgHeight) ? (int)(g_currentImgHeight - startY) :
winH;

            // Копируем бит-в-бит на экран диагностической панели
            BitBlt(hdc, 0, 0, viewW, viewH, hMemDC, startX, startY, SRCCOPY);

            SelectObject(hMemDC, hOldBmp);
            DeleteObject(hBmp);
          }
          DeleteDC(hMemDC);
        }
      }
      break;


      case 4: hBrush = CreateSolidBrush(RGB(0, 50, 150));    // Режим 4 - Синий
        break;
      }
      /*
      if (hBrush)
      {
        // Описываем полезную квадратную область 400x400 под меню
        RECT rect = { 0, 0, 400, 400 };
        // Процессор закрашивает окно выбранным цветом
```

```
            FillRect(hdc, &rect, hBrush);
            DeleteObject(hBrush);
        }
        */

        // Процессор закрывает рисование
        EndPaint(hWnd, &ps);
        return 0;
    }
    break;

    case WM_CLOSE:
        ShowWindow(hWnd, SW_HIDE);
        return 0;
    case WM_DESTROY:
        return 0;
    default:
        return DefWindowProc(hWnd, message, wParam, lParam);
    }
    return 0;
}

// === 2. САМА ФУНКЦИЯ СОЗДАНИЯ ДИАГНОСТИЧЕСКОГО ОКНА С МЕНЮ ===
void CreateDiagnosticWindow(HINSTANCE hInstance, HWND hParentWnd)
{
    WNDCLASSEX wcex = {};
    wcex.cbSize = sizeof(WNDCLASSEX);
    wcex.style = CS_HREDRAW | CS_VREDRAW;
    wcex.lpfnWndProc = DiagnosticWndProc;
    wcex.hInstance = hInstance;
    wcex.hCursor = LoadCursor(NULL, IDC_ARROW);
    wcex.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
```

```
    wcex.lpszClassName = L"HDR_Diagnostic_Class";

    RegisterClassEx(&wcex);

    HMENU hMenuBar = CreateMenu();
    HMENU hPopupMenu = CreatePopupMenu();

    if (hPopupMenu)
    {
        AppendMenu(hPopupMenu, MF_STRING, 2000, L"Режим 1: Исходный");
        AppendMenu(hPopupMenu, MF_STRING, 2001, L"Режим 2: Монохромный");
        AppendMenu(hPopupMenu, MF_STRING, 2002, L"Режим 3: Сжатый в 4 раза (4x4)");
        AppendMenu(hPopupMenu, MF_STRING, 2003, L"Режим 4: Растянутый после сжатия");
        AppendMenu(hPopupMenu, MF_STRING, 2004, L"Режим 5: Разница монохромного и растянутого");
        AppendMenu(hPopupMenu, MF_STRING, 2005, L"Режим 6: Сжатый и применена гамма-коррекция");
        AppendMenu(hPopupMenu, MF_STRING, 2006, L"Режим 7: Растянутый после гамма-коррекции");
        AppendMenu(hPopupMenu, MF_STRING, 2007, L"Режим 8: Сумма той разности и растянутого после гамма-коррекции");
    }
    AppendMenu(hMenuBar, MF_POPUP, (UINT_PTR)hPopupMenu, L"Режимы");

    RECT parentRect;
    GetWindowRect(hParentWnd, &parentRect);
    int posX = parentRect.right + 10;
    int posY = parentRect.top;

    g_hDlgWnd = CreateWindowEx(
        0, L"HDR_Diagnostic_Class", L"HDR Дефектоскоп — Панель Диагностики",
        WS_OVERLAPPEDWINDOW | WS_CLIPCHILDREN | WS_HSCROLL | WS_VSCROLL,
        posX, posY, 400, 400,
        hParentWnd, hMenuBar, hInstance, NULL
```

```
    );

    if (!g_hDlgWnd) return;

    g_hDlgStatusWnd = CreateStatusWindow(
        WS_CHILD | WS_VISIBLE,              // Стандартные стили из нашего эталона 2004 года!
        L"Диагностика активна. Ожидание кадра.",  // Стартовый текст для правого окна
        g_hDlgWnd,                          // Родительское окно (наша панель диагностики)
        2006                                // Уникальный ID второго статус-бара
    );

    ShowWindow(g_hDlgWnd, SW_SHOW);

    if (g_hDlgWnd)
    {
        HMENU hDlgMenu = GetMenu(g_hDlgWnd);
        if (hDlgMenu)
        {
            HMENU hModesMenu = GetSubMenu(hDlgMenu, 0); // Самое первое выпадающее меню "Режимы"
            if (hModesMenu)
            {
                // Ставим галочку на "Режим 1: Исходный" сразу при создании окна
                CheckMenuItem(hModesMenu, 2000, MF_BYCOMMAND | MF_CHECKED);
            }
        }
    }
    UpdateWindow(g_hDlgWnd);
}
```